FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Raven Gale N. Enriquez
Section:	AN31
Professor:	Ms. Crisola G. Tan
Date:	Oct 2, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice

- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule

3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]

```
Rule-Based Reasoning Actions:
- Turn on the aircon
- Bring umbrella
```

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?
2. Which is more flexible in solving unstructured problems?
3. Which is better for situations with continuously changing data?
4. Examples of real-world applications for both.

Your Code - Laboratory Tasks ⇩

```
# ==================================
# PART 1 - KNOWLEDGE REPRESENTATION
# ==================================
facts = {
```

```
    "temperature": 32,
    "is_raining": True,
    "humidity": 85,
    "is_night": False,
    "power_available": True,
    "room_occupied": True
}

print("=== KNOWLEDGE REPRESENTATION ===")
for category, value in facts.items():
    print(f"{category}: {value}")

# ============================
# PART 2 - RULE-BASED REASONING
# ============================
def rule_hot(f):
    if f["temperature"] > 30 and f["power_available"]:
        return "Turn on the aircon"

def rule_rain(f):
    if f["is_raining"]:
        return "Bring an umbrella"

def rule_humidity(f):
    if f["humidity"] >= 80 and f["room_occupied"]:
        return "Use a dehumidifier"

def rule_night(f):
    if f["is_night"] and f["room_occupied"]:
        return "Turn on room lights"

rules = [rule_hot, rule_rain, rule_humidity, rule_night]

actions = []
for rule in rules:
    result = rule(facts)
    if result:
        actions.append(result)

print("\n=== RULE-BASED REASONING ACTIONS ===")
for action in actions:
    print(f"- {action}")
```

```
# ==============================
# PART 3 - CASE-BASED REASONING
# ==============================
case_base = [
    {
        "problem": {
            "temperature": 31,
            "is_raining": True,
            "humidity": 82
        },
        "solution": "Turn on aircon, bring an umbrella, and use a dehumidifier."
    },
    {
        "problem": {
            "temperature": 24,
            "is_raining": True,
            "humidity": 75
        },
        "solution": "Bring an umbrella; no cooling action is needed."
    },
    {
        "problem": {
            "temperature": 33,
            "is_raining": False,
            "humidity": 60
        },
        "solution": "Turn on the aircon and drink water regularly."
    },
    {
        "problem": {
            "temperature": 28,
            "is_raining": False,
            "humidity": 88
        },
        "solution": "Use a dehumidifier and keep the room ventilated."
    }
]

new_case = {
    "temperature": facts["temperature"],
    "is_raining": facts["is_raining"],
    "humidity": facts["humidity"]
}
```

```
def cbr_similarity(new_problem, old_problem):
    """
    Count matching features.
    Each of the three features contributes one point.
    """
    score = 0
    for feature in new_problem:
        if new_problem[feature] == old_problem[feature]:
            score += 1
    return score

scored_cases = []
for case in case_base:
    score = cbr_similarity(new_case, case["problem"])
    scored_cases.append((score, case))

scored_cases.sort(key=lambda item: item[0], reverse=True)
best_score, best_case = scored_cases[0]

print("\n=== CASE-BASED REASONING ===")
print("Similarity ranking:")
for score, case in scored_cases:
    print(f"{score}/3 -> {case['problem']}")

print("\nRetrieved case:")
print(best_case["problem"])
print(f"Similarity: {best_score}/3")
print(f"Reused solution: {best_case['solution']}")

# Revise step: tailor the solution to the current temperature/humidity.
revised_solution = best_case["solution"]
if new_case["temperature"] >= 32 and new_case["humidity"] >= 80:
    revised_solution = (
        "Turn on the aircon, bring an umbrella, and use a dehumidifier."
    )

print("\nRevised solution:")
print(revised_solution)

case_base.append({
    "problem": new_case,
    "solution": revised_solution
```

```
})

print("\nRetain step:")
print(f"New case saved. Total cases: {len(case_base)}")
```